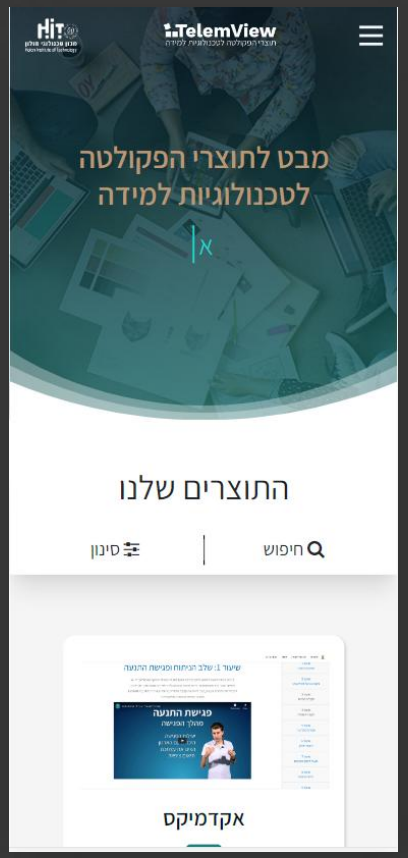
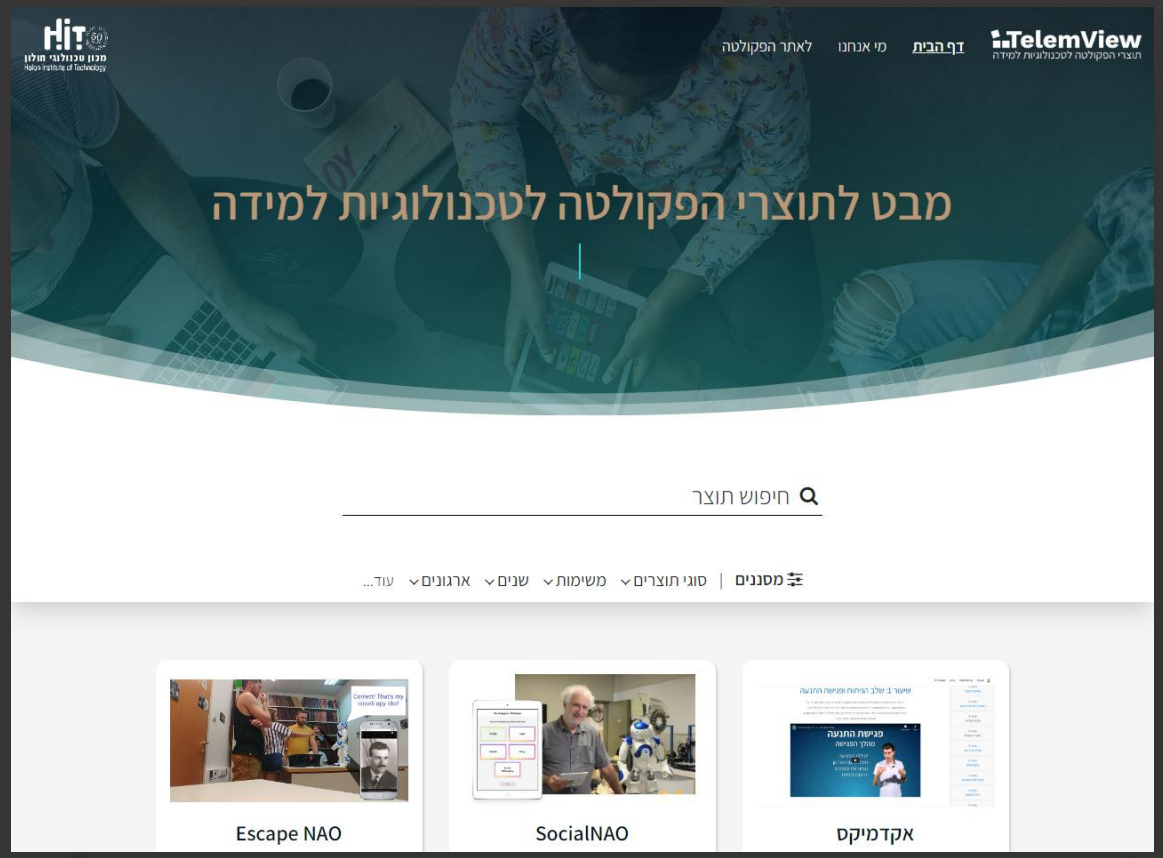
תכנות 3

רספונסיביות באתרי רשת וBootstrap

רספונסיביות

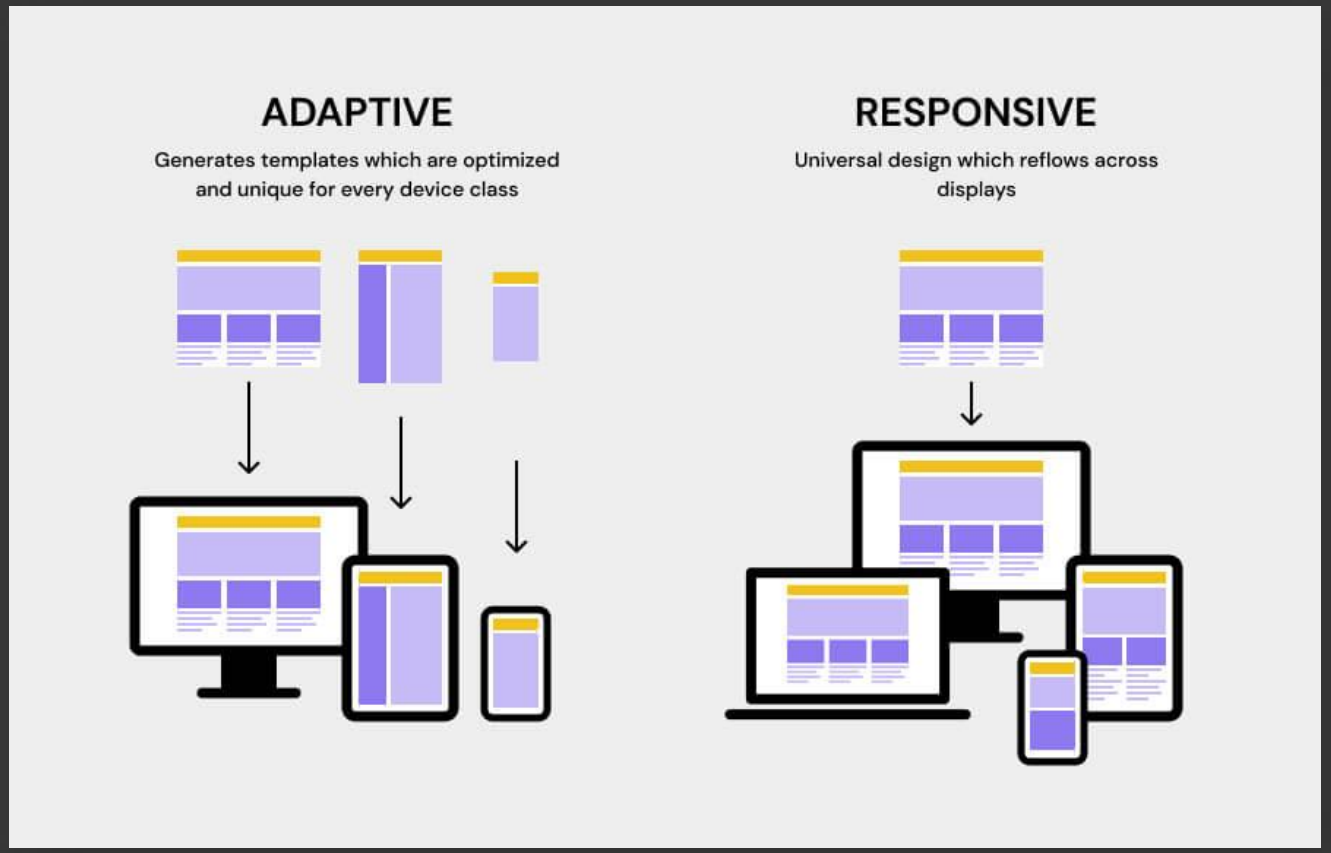
- גישה לפיה עיצוב ופיתוח אתר צריך להגיב להתנהגות המשתמש ולסביבתו בהתאם לגודל המסך, הפלטפורמה והכיוון (אוריינטציה).





https://telemview.telem-hit.net/home

רספונסיבי VS אדפטיבי



- **אתר אדפטיבי** הוא אתר שמראש מעוצב ומפותח למספר פלטפורמות שונות וקבועות.
- **עיצוב רספונסיבי** יהיה בנוי מפריסה אחת שתשתנה בהתאם לגודל המסך.

- דוגמאות לאתרים:
- אדפטיבי: https://www.ynet.co.il/home/0,7340,L-8,00.html
- רספונסיבי:
- https://wedding.s-tmro.com/
- https://taismo.de/

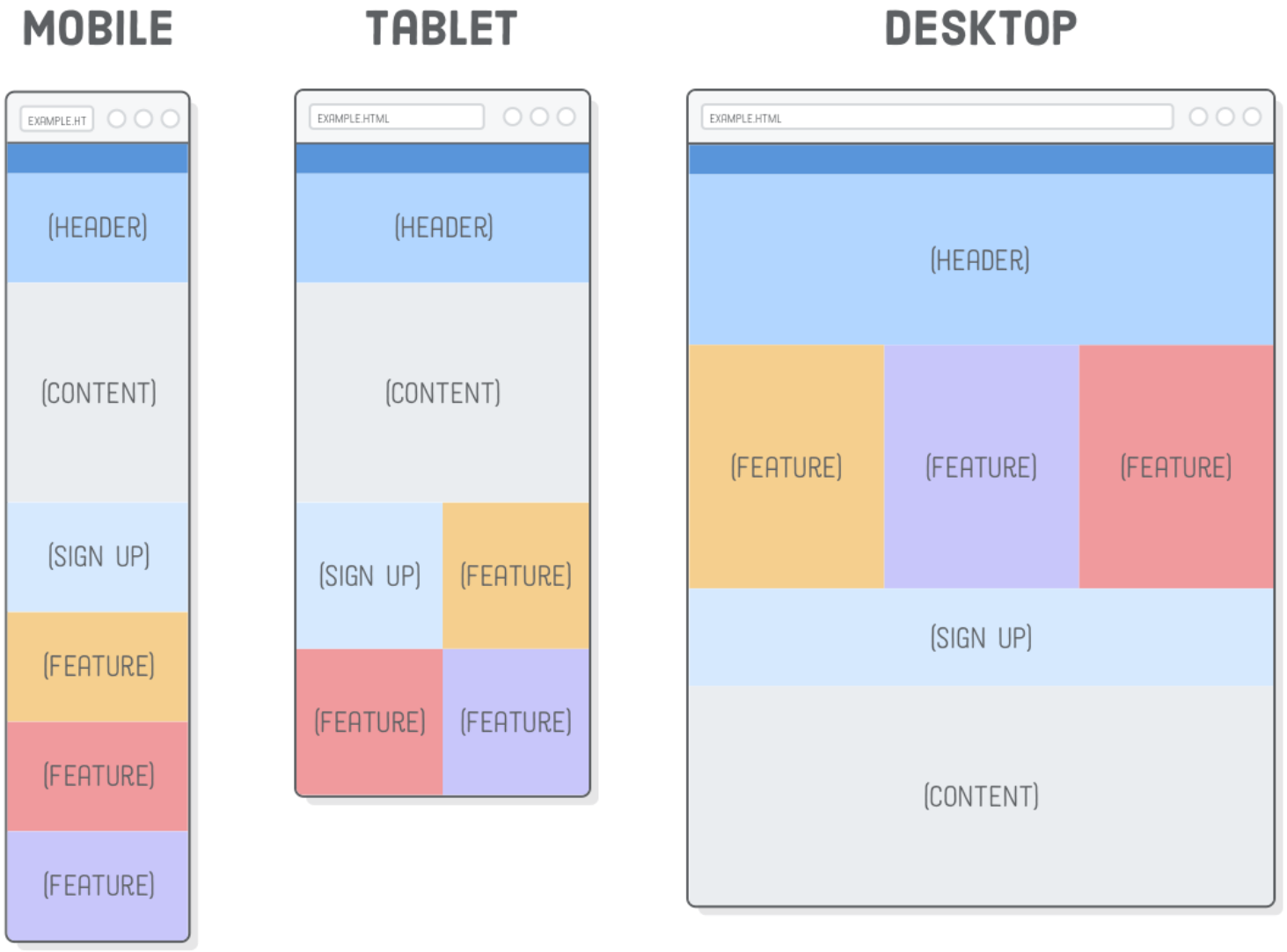
דוגמה לאתר אדפטיבי





https://www.ynet.co.il/

דוגמה לגריד של אתר רספונסיבי



תגית viewport

- viewport הוא החלק של דף האינטרנט שהמשתמש רואה.
- ה viewport משתנה בהתאם לגודל המסך.
- נגדיר בתוך תגית הhead של כל עמוד את ה viewport כך שהדף יותאם אוטומטית לגודל המסך ובנוסף שה zoom של המסך תמיד יהיה 100% בעת עליית הדף.

```
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
```

תגית viewport

התפריט

במסך מחשב: נרצה שהתפריט יהיה בשורה אחת בראש הדף.
במסכים קטנים: נרצה שהתפריט יופיע כאייקון וייפתח בלחיצה.

איך נעשה זאת? נשלב מספר טכניקות:
Flexbox – לעיצוב הפריטים בשורה אחת
Media Query – לשינוי הנראות בעת מעבר בין גדלי מסך

Flexbox

בקורס פיתוח אתרים למדנו שימוש בdisplay כגון: block, inline-block, inline ו- float בשילוב עם positions כדי לשלוט במיקומים של אלמנטים שונים בדף.

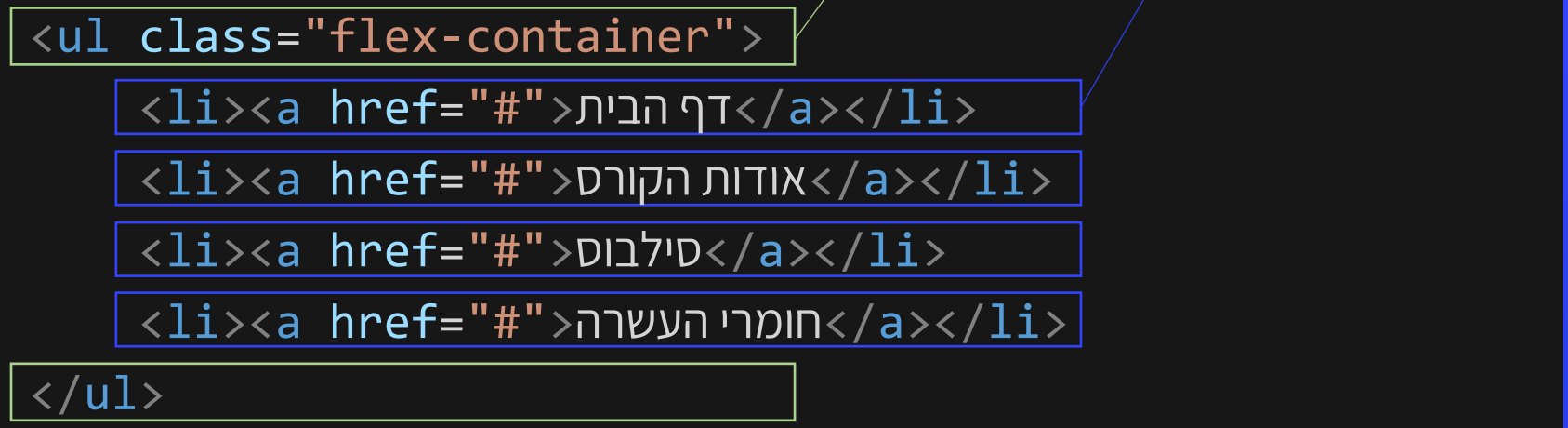
שימוש Flexbox יעזור לנו לבצע את כל הפעולות האלו בקלות ויקל על פיתוח אתר רספונסיבי.

Flexbox
איך זה עובד?

Flexbox עובד באמצעות שני חלקים:

container •

Items •



Flexbox
איך זה עובד?

ב CSS נגדיר לcontainer את ההגדרות הבאות:

```
.flex-container
{
    display: flex;
    justify-content: flex-start;
    align-items: center;
    background-color: ##051E32;
}
```

Flexbox
איך זה עובד?

ניתן להגדיר גם מגוון פרמטרים עבור Items, בעוד שהגדרה ברמת ה container תשפיע על הבנים הישירים שלו, הגדרה ברמת ה item תשפיע רק על הפריט הספציפי.

לדוגמה - נרצה להגדיר שהפריט הראשון גדול פי 4 מהאחרים:

```
.flex-container > li {
      flex-grow: 1;
      }
.flex-container > li:first-child{
      flex-grow: 4;
}
```

Media query

שיטה להגדרת CSS רק כאשר **תנאי מסויים מתקיים**.
נשתמש בעיקר על מנת ליצור עיצוב משתנה בהתאם לגודל המסך.
נעבוד בגישת mobile-first: העיצוב הראשי יהיה למובייל ונוסיף break-points בהתאם לגודל המסך.

```css
/* Extra small devices (phones, 600px and down) */
@media only screen and (max-width: 600px) {
    ...
}

/* Small devices (portrait tablets and large phones, 600px and up) */
@media only screen and (min-width: 600px) {
    ...
}

/* Medium devices (landscape tablets, 768px and up) */
@media only screen and (min-width: 768px) {
    ...
}

/* Large devices (laptops/desktops, 992px and up) */
@media only screen and (min-width: 992px) {
    ...
}

/* Extra large devices (large laptops and desktops, 1200px and up) */
@media only screen and (min-width: 1200px) {
    ...
}
```

Media query

Media query

בהתאם לגישת mobile first נגדיר את התפריט שלנו.

נרצה שרק במסכים ברוחב גדול מ 768px התפריט יוצג בשורה אחת בראש העמוד ובמסכים קטנים בטור.

לכן נשנה את הCSS כך:

```
 .flex-container {
      flex-direction: column;
   }
@media only screen and (min-width: 768px) {
   .flex-container {
      flex-direction: row;
   }
}
```

Media query

בנוסף, במסכים קטנים נרצה שהתפריט יופיע רק בלחיצת כפתור וייסגר בלחיצה נוספת. לצורך כך נצטרך לקבוע מחלקה להסתרה של האובייקט במובייל ובנוסף להגדיר לו id ייחודי:

```
<ul id="main-nav" class="flex-container hide-mobile">

.flex-container.hide-mobile {
      display: none;
  }

@media only screen and (min-width: 768px) {
.flex-container.hide-mobile {
      display: flex;
  }
}
```

Media query

נוסיף כפתור בHTML ונדאג להסתירו במסכים גדולים:

```
<button id="nav-button">תפריט</button>

#nav-button{
    display: block;
}

@media only screen and (min-width: 768px) {
    #nav-button{
        display: none;
    }
}
```

Media query

לבסוף, נוסיף פונקציית javascript שבלחיצה על הכפתור תסיר או תוסיף את המחלקה שמסתירה את התפריט:

```
<button id="nav-button" onclick="toggleNav()">תפריט</button>

function toggleNav(){
document.getElementById("main-nav").classList.toggle("hide-mobile");
}
```

תמונות ווידאו

- כל עוד לא נגדיר אחרת, מדיה שנציג באתר תשמור תמיד על גודלה המקורי
- במסכים קטנים, המדיה תצא מגבולות המסך ותשבש את הנראות
- הפתרון: הוספת רוחב מקסימלי לפריטי המדיה

```css
img {
    max-width: 100%;
    height: auto;
}
```

תמונות ווידאו

טקסט

- נרצה ליצור ממשק "חכם" בו גודל הטקסט משתנה בהתאם לגודל המסך
- לצורך כך נגדיר לכל גודל מסך את גודל הטקסט הבסיסי אליו נתייחס
- נשתמש ביחידות מידה מסוג rem שיתייחסו לגודל הפונט שהגדרנו עבור גודל המסך

```css
html {
  font-size: 16px;
}
h1{
  font-size: 2rem;
}
p{
  font-size: 1rem;
}

@media only screen and (min-width: 768px) {
    html{
        font-size: 24px;
    }
}
```

טקסט

יחידות מידה

יחידה	משמעות
px	פיקסלים
em	יחסי לגודל הפונט של האלמנט (1em שווה לפעם אחת גודל הפונט)
rem	יחסי לגודל הפונט של האלמנט הראשי (root) (1rem שווה לפעם אחת גודל הפונט)
vw	1% מרוחב המסך
vh	1% מגובה המסך
%	האחוז מגודל אלמנט האב

**בעיצוב רספונסיבי נעדיף שימוש ביחידות רלטיביות ולא ביחידות אבסולוטיות.

כך נבטיח שינוי אוטומטי של אלמנטים בהתאם לשינוי גודל המסך.